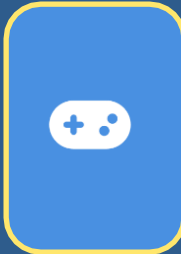


클라이언트 개발자 기술 설명서

Click on your user name to begin

A blue rounded square icon with a white game controller symbol inside.

User 이민혁

Type your password

A green circle with a white right-pointing arrow.A grey circle with a white question mark.



Turn off computer

To add or change accounts go to Control Panel.

파일(F) 편집(E) 보기(V) 도구(T) 도움말(H)

←

→

↑

✂

📁

📁

주소(D) C:\Portfolio\Projects\WinAPI_DontStarve

WinAPI_DontStarve 모작

플랫폼	PC(Windows)		
개발 인원	1인(개인)	개발 기간	2025. 12. 05 ~ 2026. 02. 04 (대략 2달)
개발 도구	C++, WinAPI	동영상 링크	유튜브 링크
요약	플레이어는 마우스 조작을 통하여 이동, 상호작용을 할 수 있으며, 최종 목표인 최종 보스 몬스터를 쓰러트리는 것을 목적으로 한다. 최종 보스 몬스터를 쓰러트리기 위해서 플레이어는 재화를 수집하여, 한정된 자원으로 아이템을 만들어 보스에 도전할 수 있다.		
동기	게임 엔진을 의존하지 않고, WinAPI를 이용하여 완전 밑바닥부터 게임을 만들어보고자 모작에 도전하게 되었다. 게임 실행의 전체적인 파이프라인을 파악하고, 이를 스스로 설계해 보고자 하여 시작하게 되었다.		

DonStarve_Client

진행상황 초기화

게임시작

종료

시작

탐색 중 - Projects

그리드 공간 분할 컬링 설계

문제 상황

맵에 배치되는 오브젝트가 1,000~10,000개로 늘어남에 따라 뷰포트 내 검사 후보를 찾기 위한 루프 비용 과다 발생.

뷰포트만 아니라 오브젝트의 콜라이더 검사, 상호작용 처리 로직에서도 **수 많은 객체의 루프에 대한 부담**이 생긴다.

관련 코드

```
for (GameObject* obj : m_worldObjects) {  
    if (!obj || !obj->IsEnabled() || obj->IsDead())  
        continue;  
  
    if (cameraManager->IsObjectInViewPort(obj))  
    {  
        outObjects.push_back(obj);  
    }  
}
```

모든 월드 오브젝트를 순회하여
뷰포트안에 존재하는지 확인

관련 이미지



월드 오브젝트 개체가 많아질 수록
순회의 부담감이 증가

그리드 공간 분할 컬링 설계

설계 내용

먼저 전체 월드 공간을 일정한 크기인 GRID_CELL_SIZE 단위의 **격자(Grid Cell)**로 **분할**하고,
각 게임 오브젝트의 중심 좌표를 기준으로 소속 격자를 계산하여
2차원 공간 배열(m_spatialGrid)에 할당, 관리한다.

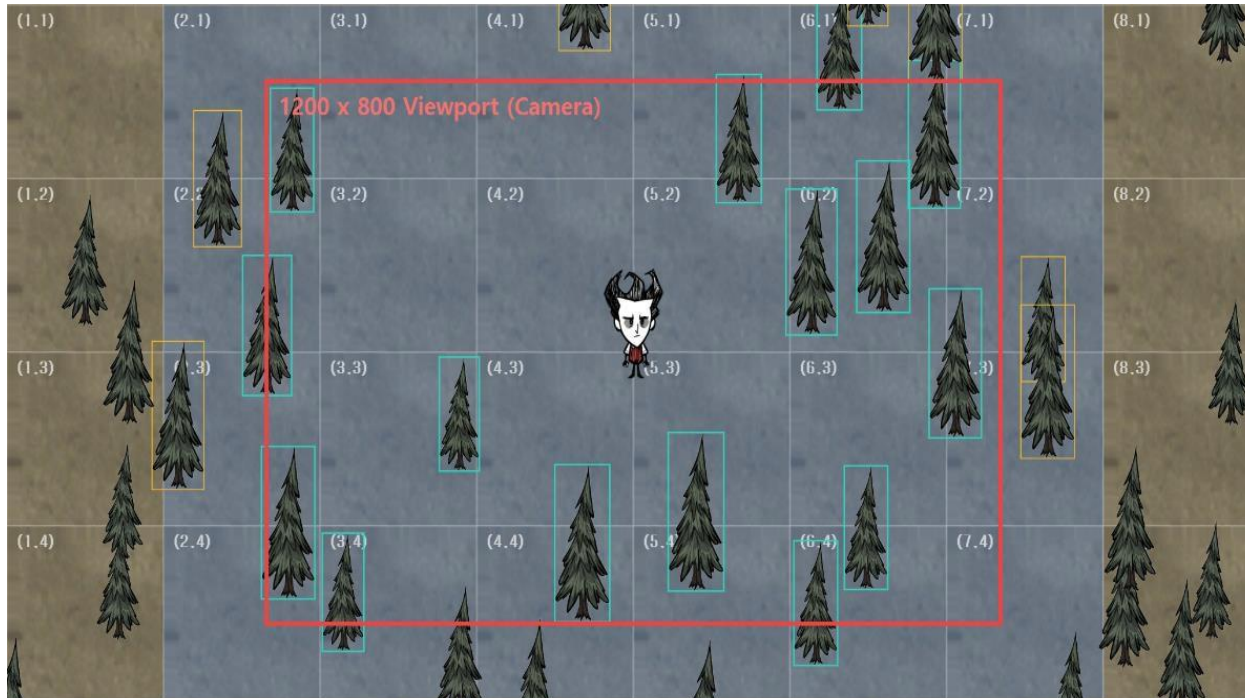


그림 예시

해당 그림에서 GRID_CELL_SIZE는 256, 타일 크기는 128 x 128이며, 1200 x 780 윈도우 화면(뷰포트 크기)의 구성으로 이뤄진다.

- 활성 오브젝트
- 후보군 오브젝트
- 비활성 오브젝트

파일(F) 편집(E) 보기(V) 도구(T) 도움말(H)

← → ↑

✂ 📄 📁

주소(D)

C:\Portfolio\Projects\File Tower Defence

File Tower Defence

플랫폼	PC(Windows)	장르	로그 라이크 타워 디펜스
개발 인원	4인 (기획1,플밍2,아트1)	개발 기간	2025. 05 ~ 현재
개발 도구	C#, Unity	동영상 링크	유튜브 링크
요약	Windows OS와 파일, 확장자를 컨셉으로 하여, 주어진 경로에 바이러스를 막는 디펜스 게임이다. 플레이어는 라운드를 클리어하여 얻은 재화를 통해 더 상위인 파일을 해금하거나 영구적으로 파일들의 위력을 높일 수 있다.		
역할	- 파일/바이러스/UI 입력 및 상호작용 이벤트 시스템 설계 - 버프 파일 유닛의 버프 부여/해제 로직 설계 - 그리드 기반 오브젝트 배치 시스템 설계		



UI 기반 설계로 인한 월드 좌표계와의 변환이 까다로운 문제점

문제 설명

1. 개발 모티브

초기 개발 단계에서, 우리 Windows OS에 존재하는 파일의 드래그&드랍의 배치 느낌을 모티브를 삼았다.

2. 초기 개발 설계

실제 파일들이 UI이기도 하니까 파일 유닛을 “UI Slot에 배치하면 되지 않을까?” 라는 생각을 하고 만듦

3. 실질적인 문제점

바이러스가 UI(RectTransform) 객체의 위치를 비교하거나 찾고자 할 때

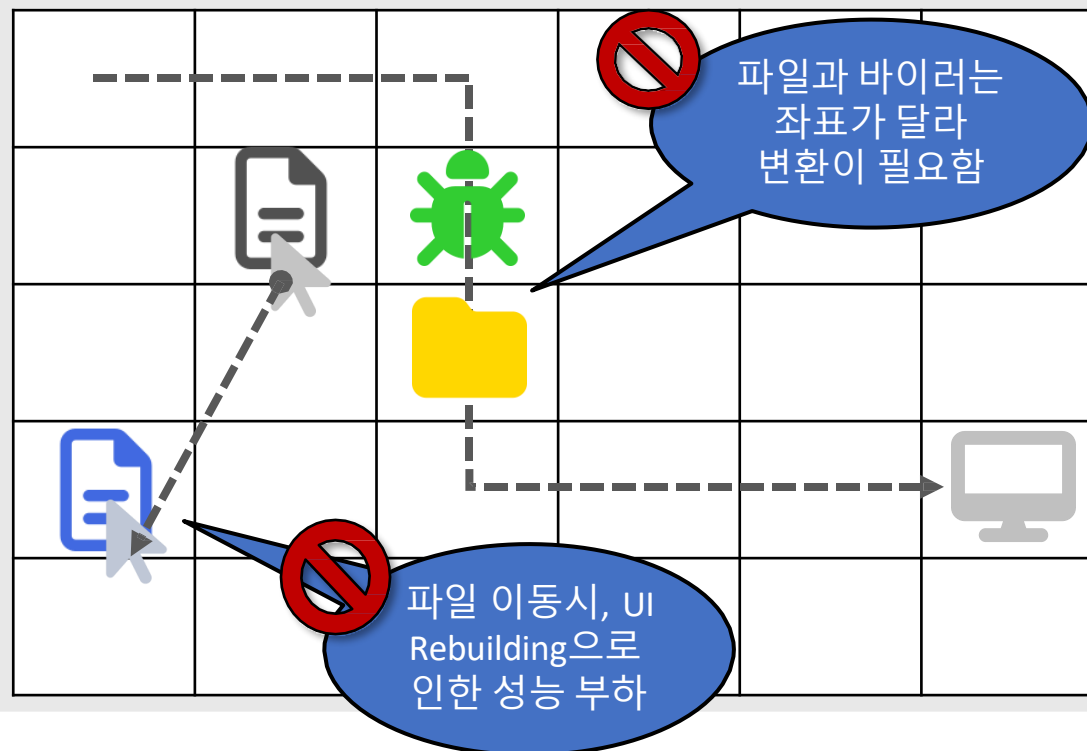
빈번한 좌표계 변환으로 로직의 복잡도가 올라갔음

다른 Canvas나, 부모 RectTransform로 이동했을 때 그리고 Canvas 안에 수 많은 UI 자식들이 존재했을 때

UI ReBuilding으로 인한 성능적인 부하도 존재함

예시 자료

💡 좌표계가 다르므로 변환은 필수적임



Grid 방식의 오브젝트 구조 변환 설계

⚙️ GameObjectGridLayout

그리드 오브젝트를 생성하고 정렬하는 것을 도와주는 클래스
그리드 사이 간격과, 크기, 행과 열 개수를 자유롭게 지정할 수 있음

📁 FileGridManager

그리드 관리자
특정 조건이나, 상황에 따른 그리드를 찾을 수 있도록 도와주는 헬퍼 함수 제공

📄 FileGrid

파일 유닛을 담아 관리하는 그리드 클래스
파일 유닛의 점유와 해제를 담당함 버퍼에 대한 정보를 보유.

STEP 01 · GameObjectGridLayout

- 행·열 개수(rows, cols) 및 셀 크기·간격 산출
- 그리드 인덱스 → 월드 스페이스 좌표 변환 수식 제공

```
// 그리드 규격 산출 & 월드 좌표 변환
Vector2 cellSize, spacing;
public Vector3 CalcCellPos(int r, int c) =>
    origin + new Vector3(c * (cellSize.x + spacing.x),
                        -r * (cellSize.y + spacing.y), 0);
```

규격 파라미터
주입 전달

STEP 02 · FileGridManager

- 2D 인메모리 배열(gridArray[,]) 공간 분할 캐싱
- 3x3 국소 인접 셀 O(1) 초고속 거리 매칭 탐색

```
// 2D 배열 기반 3x3 O(1) 탐색
FileGrid[,] gridArray = new FileGrid[rows, cols];
for (int dx = -1; dx <= 1; dx++)
    for (int dy = -1; dy <= 1; dy++)
        SearchClosest(gridArray[x + dx, y + dy]);
```

2D 인덱스
참조
& 점유 제어

STEP 03 · FileGrid

- 유닛 배치/이동 시 원자적 점유(SetOccupied) 및 롤백
- 인접 타일 버퍼 소스 컬렉션 관리 및 스택 갱신

```
// 원자적 유닛 점유 & 버퍼 동기화
HashSet<File_Base> buffSources;
public void SetOccupied(File_Base unit)
{
    CurrentUnit = unit;
    UpdateBuffs(buffSources);
}
```

월드 좌표로 변환한 결과



그리드 공간 별로
유닛 배치 및 영역 밖에 있는
파일 유닛은 근접한 그리드로
점유 하도록 설계

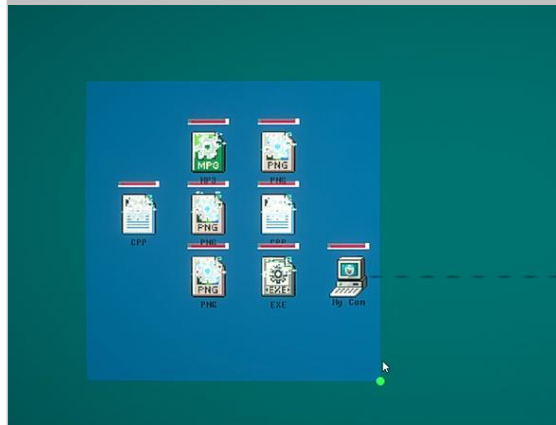
이 결과 Windows 바탕화면
아이콘을 드래그&드랍하는
느낌을 유지할 수 있었음



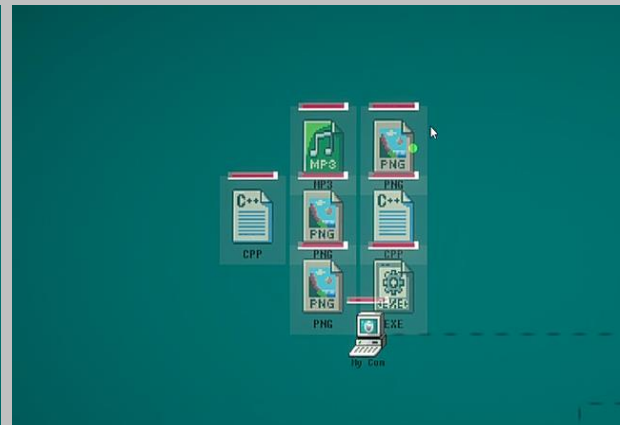
유연한 그리드 설정 수정, 그
리드 셀 크기 자동 조정 기능
을 가진 레이아웃 클래스

해당 레이아웃으로 생성한
그리드들(초록색)

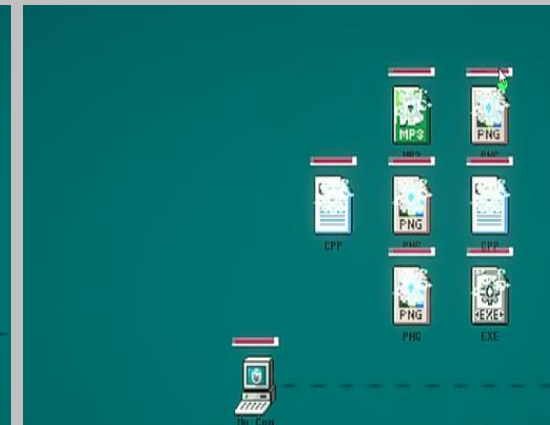
결과물(Result)



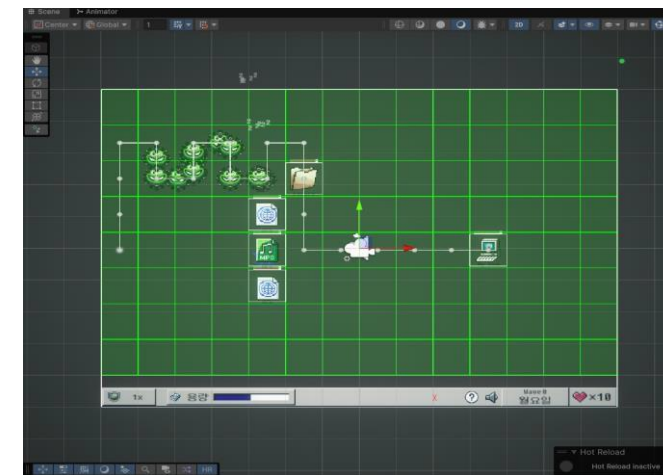
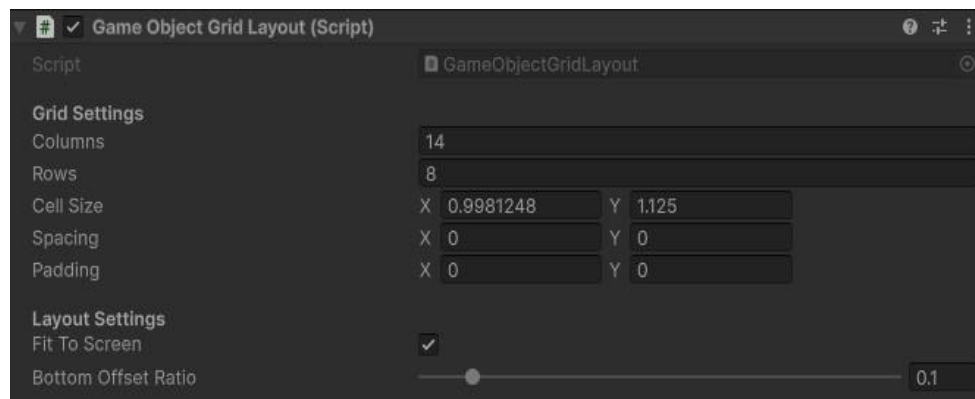
1단계 파일 객체 선택



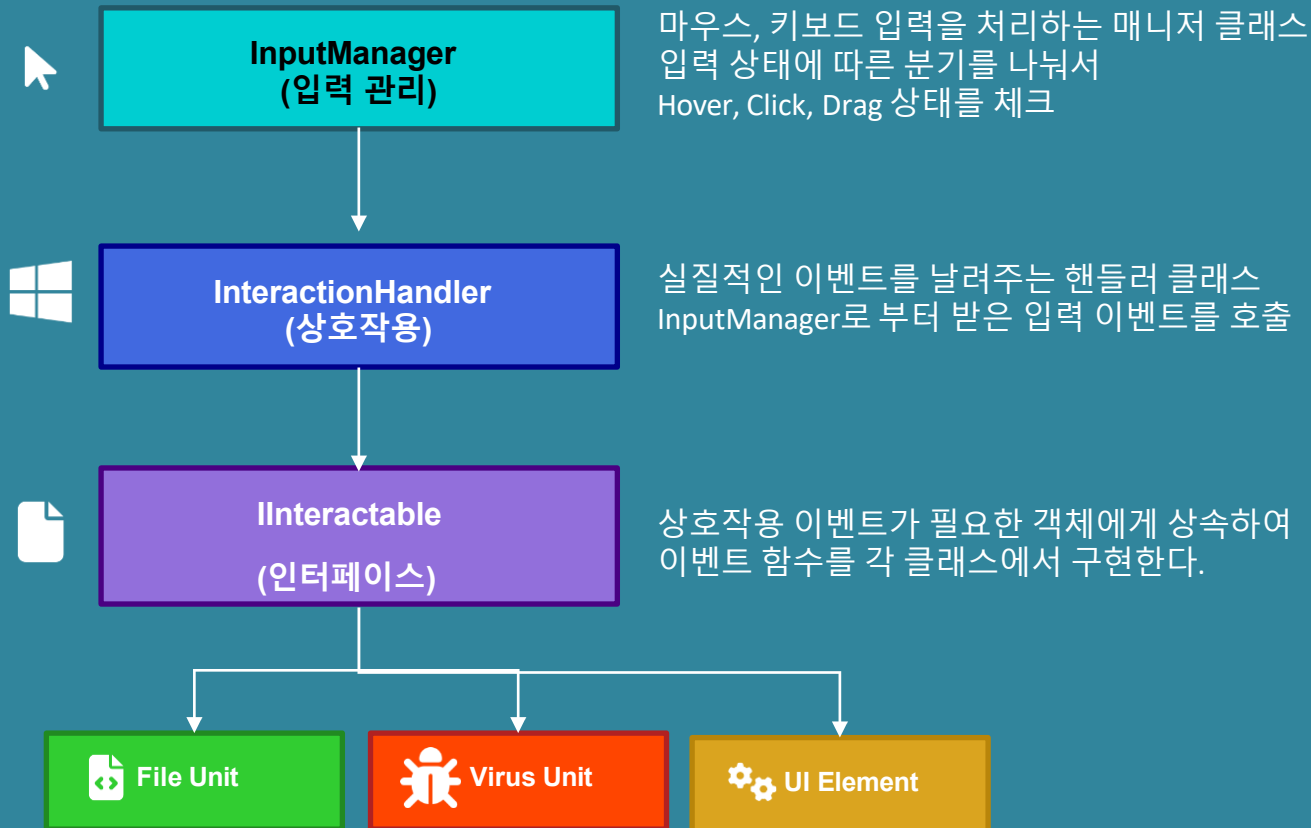
2단계 파일 객체 드래그



3단계 원하는 위치에 배치



입력 이벤트 처리 시스템



```
public interface IInteractable
{
    // 이벤트 메서드
    void OnHoverEnter(); // 마우스가 객체 위로 올라올 때
    void OnHoverExit(); // 마우스가 객체에서 벗어날 때
    void OnClickEnter(); // 마우스 버튼을 누를 때
    void OnClickExit(); // 마우스 버튼을 땔 때
    void OnBeginDrag(); // 드래그 시작
    void OnDrag(Vector2 mouseDelta);
    void OnEndDrag(); // 드래그 종료
    void OnClick(); // 클릭 처리
    void OnDoubleClick(); // 더블클릭 처리
    void OnRightClick(); // 우클릭 처리

    void OnSelected(bool isSelected);
}
```

파일(F) 편집(E) 보기(V) 도구(T) 도움말(H)

←

→

↑

✂

📄

📁

주소(D)

C:\Portfolio\Projects\World First Kill

World First Kill

플랫폼	모바일(Android)	장르	RPG, 시뮬레이션
개발 인원	9인(기획1, 아트4, 플밍4)	개발 기간	2025.03 ~ 2025.09
개발 도구	C#, Unity	동영상 링크	유튜브 링크
요약	매 한 판 한 판이 달라지는 RPG의 재미를 줄 수 있는 게임이다. 매 판마다 등장하는 맵, 퀘스트, 직업 정보가 달라지므로, 이런 상황적 랜 덤성을 이용해 플레이어는 상황에 맞는 전략과 빌드를 선택해서 게임을 즐길 수 있다.		
동기	- 서버로부터 받아온 CSV 데이터 관리/파싱 시스템과 CSV 버전 관리 - 퀘스트 수락과 보상, 클리어 시스템 - 맵 이동 시스템 - Seed/Token 기반 세이브/로드 시스템		

시작

탐색 중 - Projects

Json기반 저장의 부담감에 대한 문제에 대한 고민

상황 분석 및 고민

던전, 직업 스킬 및 조합, 퀘스트, 직업 정보를 다 **Json**으로 저장하면 데이터 크기가 커지고 관리하기 번거로울 수 있음
방대한 Json을 해석하고, 이를 게임에 적용하기에 파싱하는 과정에서 오버헤드나, 코드의 복잡성이 올라갈 수 있다.

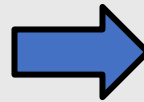
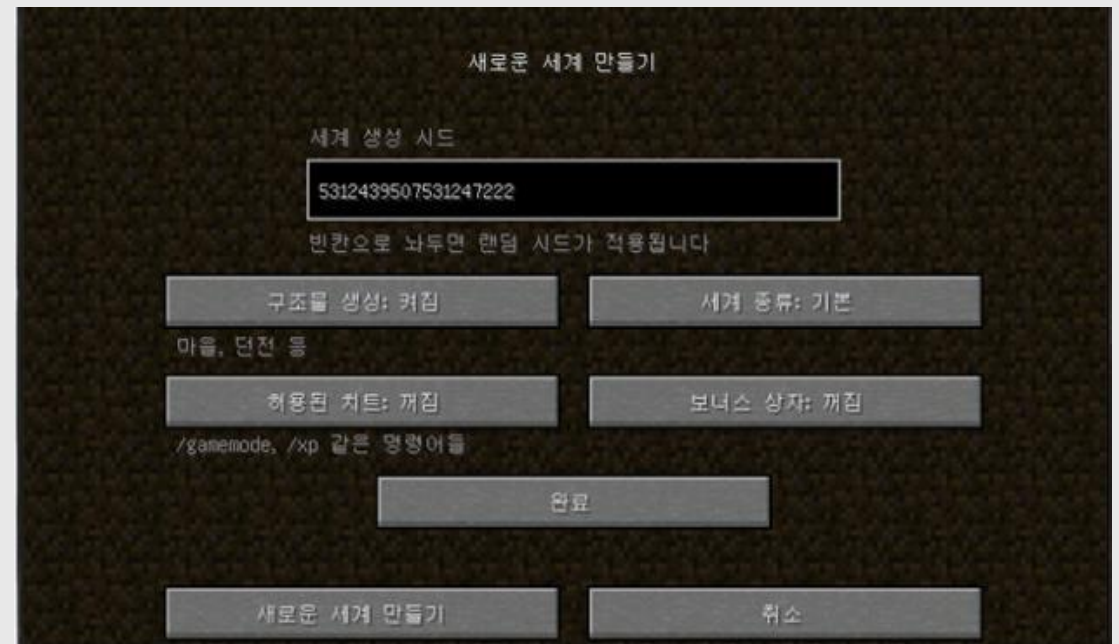
Seed 기반 생성 아이디어 도출

마인크래프트의 Seed형 맵 생성을 떠올려,
Seed/Token 형식의 맵 생성, 퀘스트 생성과 보상값
플레이어 직업과 클래스를 생성해 보자

마인크래프트에서는 해당 시드로, 지형, 환경, 구조물 등을
하나의 코드형태로 생성할 수 있으며,
똑같은 시드를 불러오면
그 맵의 정보를 그대로 가져와 똑같이 만들어 줄 수 있다.

**핵심은 하나의 시드로 세이브/로드
가능하게 하는 것이다.**

💡 마인크래프트라는 게임을 참고하여 Seed 시스템 아이디어 구상



Seed기반 난수 생성 과정



GameSeed

BaseSeed: 111111

```
public enum Domain : uint
{
    Map = 1,
    Region = 2,
    Field = 3,
    Quest = 4,
    Dialog = 5,
    Reward = 6,

    Class_Melee = 100,
    Class_Dexterity = 101,
    Class_Magic = 102,
    Class_Support = 103,
    Skill_Melee = 200,
    Skill_Dexterity = 201,
    Skill_Magic = 202,
    Skill_Support = 203,
}
```



GameSeed – SplitMix64 기반 함수

BaseSeed: 111111 + Domain enum값

도메인별 새로운 난수값 생성 및
딕셔너리에 저장

```
/// <summary>
/// 서브 시드 파생 (SplitMix64 유사 혼합, 32-bit 출력)
/// </summary>
private static int DeriveSubSeed(long baseSeed, uint domain)
{
    ulong z = (ulong)baseSeed + 0x9E3779B97F4A7C15UL + ((ulong)domain * 0x85EBCA6BUL);
    z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9UL;
    z = (z ^ (z >> 27)) * 0x94D049BB133111EBUL;
    z = z ^ (z >> 31);
    return (int)(z & 0xFFFFFFFF); // 양의 정수로 고정
}
```

SplitMix64를 사용하여

Map, Quest, Class, Skill 등
난수 사용량 기록생성된 난수값을 이용해
서 각 도메인에 대한 난수
값을 사용 시 기록한다.

랜덤 생성

도메인별
난수 사용량 증가해당 난수 사용량을 저장
하기
위해서 딕셔너리에 저장
한다.

1826533749 Seed 예시로 결과 살펴보기

데이터 인코딩 & JSON 예시

```
{  
  "baseSeed": 1826533749,  
  "seedToken": "0000000000182653374901400100...",  
  "updatedAt": "2026-05-19 21:20:54"  
}
```

DATA ENCODING MAP

[BASE SEED]

[COUNT]

[ID]

[USAGE PROGRESS...]

00000000001826533749 (20자)

맵과 규칙을 생성하는 고유 Seed번호

003 (3자)

저장된 도메인(맵, 퀘스트, 스킬 등)의 개수

001 (3자)

도메인 값

000000000042 (12자)

해당 시스템의 난수 사용 누적량

결과 요약



SeedToken 완벽 복원

하나의 SeedToken을 이용해서 게임 내 난수 기능을 완벽히 불러올 수 있었다



64비트 BaseSeed 기반

총 64비트 단위의 BaseSeed를 이용하여 안정적인 세이브 및 로드가 가능해졌다



대용량 Json 저장 최소화

일일이 데이터를 저장해서 Json 파일이 커지고 관리하기 번거로워지는 문제를 완벽히 예방가능하다.

문제 Seed비교 예시로 데이터 비교

Seed A (1459589277)

Player 1



스텝 **스킬**

I   10

II   

스킬 이름

아군에게 스킬 사용 시 아군의 치명타 확률과 치명타 데미지를 증가시킵니다.

스킬 레벨업 사용하기

돌아가기

```
{
  "baseSeed": 1459589277,
  "seedToken": "000000000014595892770140010000000000260",
  "updatedAt": "2026-05-19 21:27:06"
}
```



시드값에 따른
결과 변화

Seed A (14595892776)

Player 1



스텝 **스킬**

I   10

II   

강타

하나의 적에게 120%의 AD 데미지를 입힌다.

스킬 레벨업 사용하기

돌아가기

```
{
  "baseSeed": 1459589276,
  "seedToken": "00000000001459589276014001000000000027002000000",
  "updatedAt": "2026-05-19 21:27:51"
}
```

✓ 각기 다른 시드 → 완전히 다른 스킬셋 분포 생성

스킬이 난수로 판정되기 때문에, 개발자가 해당 스킬 정보를 일일이 저장하지 않아도 단일 Seed와 Token의 해석을 통해 동일한 스킬 정보를 완벽하게 불러올 수 있게 된다.

